

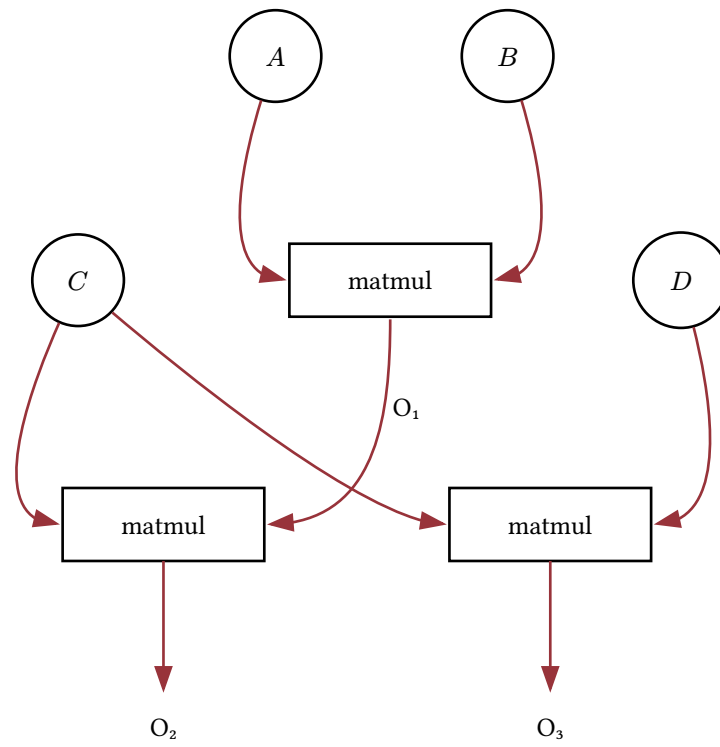
CS 521 ML & Compilers – MP 2

Part 1: Tensor Programming and Einsum Notation

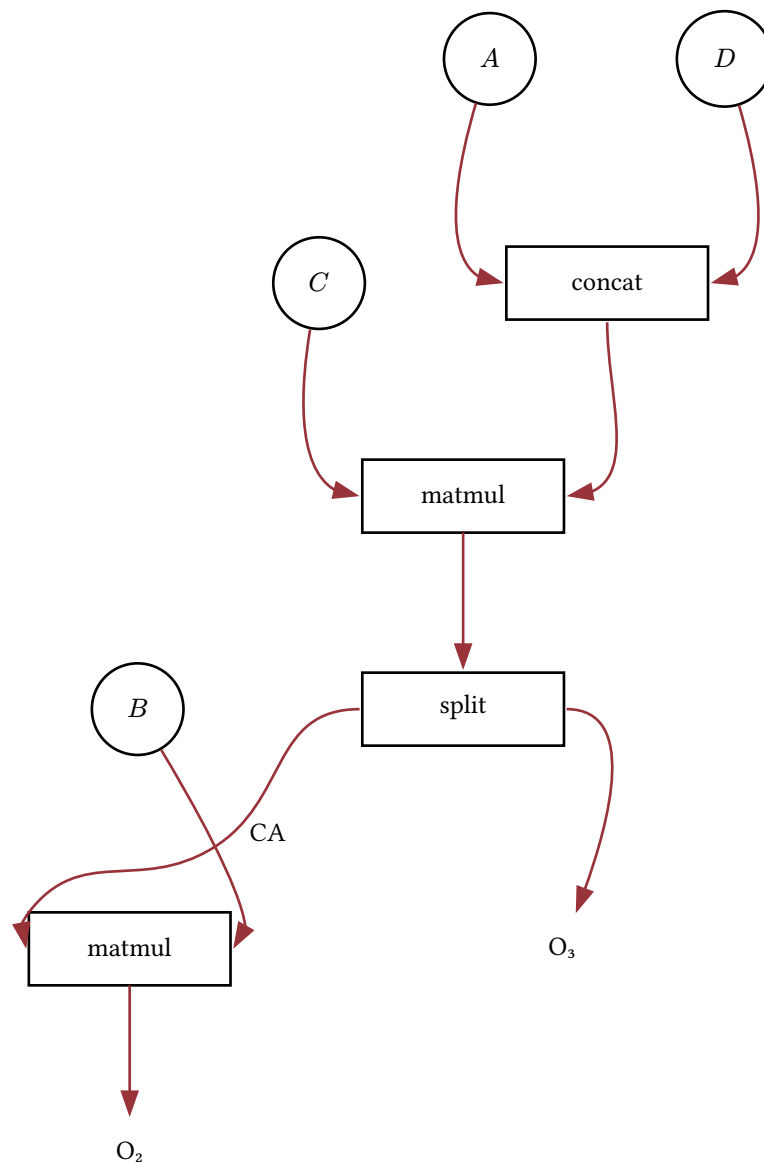
1. Reduction over one dimension (assuming a sum reduction) can be represented in einsum $ij \rightarrow j$, or equivalently $O_j = \sum_i A_{i,j}$. The index i is summed over because it does not appear in the output. Other reduction operators (e.g. maximum), are not expressible in ordinary einsum.
2. Convolution ($C_{n,m} = \sum_{k_1=0}^{K_1-1} \sum_{k_2=0}^{K_2-1} W_{k_1,k_2} \cdot B_{n+k_1,m+k_2}$) is not representable as an einsum over A and B . Einsum is capable of expressing equality of indices and summing over them but is unable to express any arithmetic combinations of them (i.e., $n + k_1$ or $m + k_2$). Thus convolution is outside the expressivity of einsum.
3. Batched matmul can be represented in einsum $bmk, bkn \rightarrow bmn$
4. Elementwise ReLU is not expressible in einsum since it requires a non-linear transformation ($\max(0, x)$) and einsum is only capable of doing multiplications and additions.
5. Hadamard products can be represented in einsum $ijk, ijk \rightarrow ijk$.

1. TCG for program

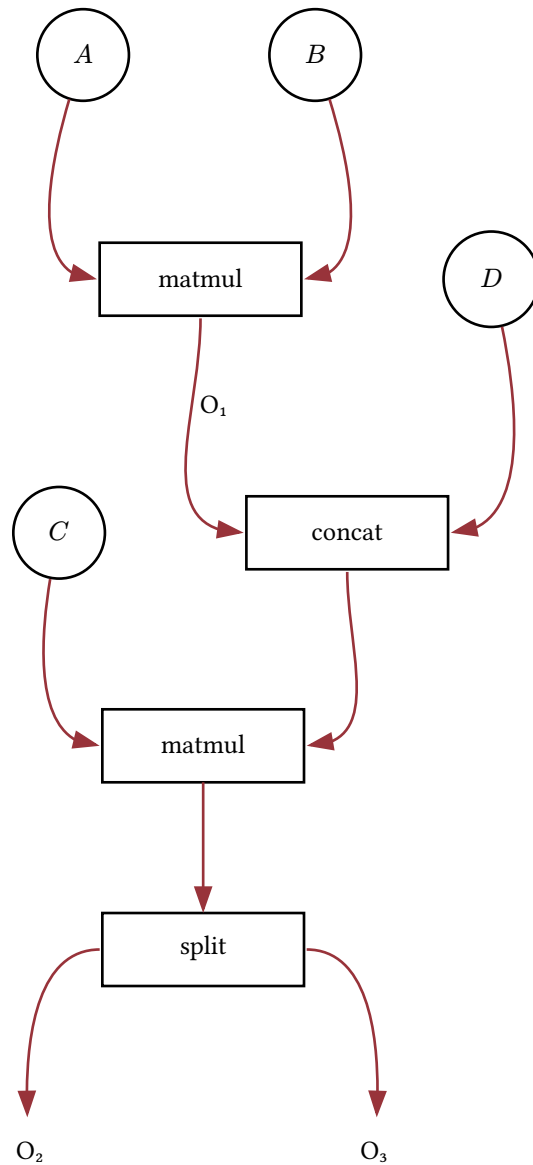
```
01 = torch.matmul(A, B)
02 = torch.matmul(C, 01)
03 = torch.matmul(C, D)
```



2. Apply rewrite R1 and then R2. R1 rewrites $C(AB)$ as $(CA)B$. R2 then combines CA and CD by concatenating A and D along their columns. The split recovers CA (left output) and $CD = O_3$ (right output); the former of which is multiplied by B to produce O_2



3. Apply rewrite R2 and then R1. R2 combines CO_1 and CD into one multiplication by C , followed by a split. $O_1 = AB$ and D are joined along their columns. R1 cannot apply in this case since the second operand of the multiplication by C is a concat.



4. Despite computing the same outputs, (b) and (c) differ in both the intermediary tensors they produce as well as how they combine them to obtain the outputs. In the first rewrite order, the shared multiplication computes $C \text{ concat}(A, D)$ and a subsequent matmul is used to compute $(CA)B$. In the second rewrite order, AB is computed first and the multiplication computes $C \text{ concat}(AB, D)$. This leaves (c) unchanged after applying rewrite rule R1 since there is no nested matmul.

5. Let

$$C : m \times k, \quad A : k \times n, \quad B : n \times l.$$

The work involving CD is the same in both rewrite orders, so it is enough to compare the cost of computing $O_2 = CAB$.

For (b), applying R1 first gives $(CA)B$. Based on the number of scalar multiplications required that cost is $mkn + mn l$

For (c), applying R2 first leaves $C(AB)$. The cost is $knl + mkl$. Therefore, (b) is more beneficial when

$$\begin{aligned} mkn + mn l &< knl + mkl \text{ or equivalently,} \\ mn(k + l) &< kl(n + m) \end{aligned}$$

Thus, applying R1 first can be more when computing the intermediary $(CA)B$ is cheaper than $C(AB)$.

